BÖLÜM 4: ASENKRON UFUKLAR — OLAY ODAKLI MIMARI

9 dk • DERS 14

Parça 14: Gözyaşı Dökmeden Kafka: Bölümler (Partitions), Offsetler ve Tüketici Grupları

Geceleri kabus görmeden veri hatlarını ölçeklemek için dağıtık olay günlüklerinin gizemini çözmek.

🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**

Bir araç çağırma platformu sürücülerin GPS konumlarını takip etmek için Apache Kafka kurdu. Konu (topic) tek bir bölüm (partition) ile açıldı. Sürücü sayısı 500'den 50.000'e çıktığında tek sunucu tıkandı. Mühendis hemen 10 yeni tüketici (consumer) sunucusu başlattı. Ancak hayretle gördü ki bu 10 sunucudan sadece BİRİ çalışıyor, diğer 9'u %0 CPU ile boş oturuyordu! Panikleyen başka bir geliştirici partition anahtarını sürücü ID'si yerine rastgele UUID yaptı. Bir anda Sürücü 42'nin GPS sinyalleri 20 farklı bölüme saçıldı. Bir yolculuk sırasıyla: 1) Sürücü ulaştı, 2) Sürücü kabul etti, 3) Yolcu indi şeklinde ters sırayla işlendi! Yolcular hiç başlamamış yolculuklar için 400 TL faturalandırıldı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Kafka geleneksel bir mesaj kuyruğu değildir; bölünmüş, çoğaltılmış, yalnızca sonuna ekleme yapılan (append-only) dağıtık bir işlem günlüğüdür. Kafka'da ölçeklenmenin temel birimi **Partition (Bölüm)**'dür ve Kafka'nın demir kuralı şudur: **Bir tüketici grubundaki aktif tüketici sayısı, konudaki partition sayısından fazla olamaz.** Paralel çalışan 10 işçi istiyorsanız en az 10 partition'ınız olmak zorundadır. Ve mesajların sırası önemliyse, aynı iş nesnesine ait olaylar mutlaka aynı partition anahtarıyla gönderilmelidir.

Temel Mimari İlkeler ve Zihinsel Modeller

Kafka'nın Temel Çarkları

- **Partition ve Anahtarlar:** Kafka `hash(anahtar) % bolum_sayisi` formülünü kullanır. `siparis_123` için tüm olaylar aynı bölüme mutlak kronolojik sırayla düşer. Tüm konu genelinde global sıra yoktur; sıra YALNIZCA aynı partition içinde garantidir.
- **Offset:** Bir partition içindeki her mesajın sıralı kimlik numarasıdır. Tüketiciler nereye kadar okuduklarını offset'lerini kaydederek (commit) takip eder.
- **Tüketici Grupları (Consumer Groups):** Birlikte çalışan işçi grubu. Bir işçi çökerse Kafka 'Rebalance' yaparak o işçinin partition'ını gruptaki başka birine atar.

Teslimat Garantileri: En Az Bir Kez (At-Least-Once)

Dağıtık sistemlerde varsayılan garanti **En Az Bir Kez**'dir. Bir tüketici mesajı işleyip offset'i kaydedemeden çökerse, yerine gelen tüketici o mesajı TEKRAR okur! Bu yüzden tüketicileriniz mutlaka idempotent (mükerrer çalışmaya dayanıklı) olmalıdır.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Rebalance Fırtınası:** Tüketici zaman aşımı sürelerini aşırı dar tutup uzun süren işlemlerde Kafka'nın işçiyi öldü sanarak sürekli partition dağıtımı yapması ve tüm sistemi kilitlemesi.
- **Rastgele Anahtar Tuzağı:** Sıralı işlenmesi gereken olaylarda partition anahtarını boş bırakıp olayların sırasını kaybetmek.

- **Devasa Mesaj Hatası:** Kafka üzerinden 20 MB'lık videolar veya dev PDF'ler taşımaya kalkışmak (Mesajda sadece S3 bağlantı adresini göndermelisiniz).

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Partition boyutlandırması: Hedeflenen veri hacmine göre partition sayısını belirle (her partition için saniyede 1-2 MB güvenli bir kuraldır).
> - **2. Adım:** Sıralı olması gereken işlerde partition anahtarını mutlaka nesne ID'si (`musteri_id`) olarak belirle.
> - **3. Adım:** Bütün olay işleyicilerini benzersiz olay ID'si (UUID) ve veritabanı benzersizlik kısıtlarıyla idempotent yap.

Mimari Çıkarımlar ve Hile Tablosu

- Kafka sıralamayı yalnızca aynı partition içinde garanti eder, tüm konu genelinde değil.
- Partition sayısından fazla tüketici ayağa kaldırmak boşuna sunucu parası ödemektir.
- Mesajların sadece bir kez geleceğine asla güvenmeyin; her mesajın iki kez gelebileceğini varsayın.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Tüketiciniz bir veritabanı kaydı attıktan hemen sonra ama Kafka'ya 'okudum' offset'ini bildirmeden önce çökerse ne olur? Kodunuz bu mükerrer okumayı güvenle atlatabilir mi?